

```

        public static float max(float a, float b, float c) {
            return Math.max(Math.max(a, b), c);
        }
    }

package nhnis.fw.common.util;

public class ObjectUtil {
    public static String toString(Object obj) {
        return (obj == null) ? "" : obj.toString();
    }

    public static String toString(Object obj, String nullStr) {
        return (obj == null) ? nullStr : obj.toString();
    }
}

package nhnis.fw.common.util;

import java.math.BigDecimal;
import java.math.BigInteger;
import java.util.Collection;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;

public class StringUtil {
    public static int countSubstring(String s, String sub) {
        return countOccurrencesOf(s, sub);
    }

    public static String toDelimitedString(Object[] arr, String delim) {
        return arrayToDelimitedString(arr, delim);
    }

    public static String toDelimitedString(Collection<?> c, String delim) {
        return collectionToDelimitedString(c, delim);
    }

    public static String[] split(String str) {
        if (str == null)
            return new String[0];
        return str.trim().split("\\s");
    }

    public static String[] split(String str, String delim) {
        if (str == null)
            return new String[0];
        if ("".equals(delim)) {
            String[] arr = new String[str.length()];
            if (null != str) {
                int strLength = str.length();
                for (int i = 0; i < strLength; i++) {
                    arr[i] = (new StringBuilder(String.valueOf(str.charAt(i)))).toString();
                }
            }
            return arr;
        }
        return delimitedListToStringArray(str, delim);
    }
}

```

```

public static boolean isEmpty(String str) {
    return !(str != null && !"".equals(str));
}

public static boolean isNumeric(String str) {
    if (str == null)
        return false;
    return isNumberOnly(str.trim());
}

public static boolean isHangul(char inputChar) {
    String unicodeBlock = Character.UnicodeBlock.of(inputChar).toString();
    return !(("HANGUL_JAMO".equals(unicodeBlock)
&& !"HANGUL_SYLLABLES".equals(unicodeBlock)
&& !"HANGUL_COMPATIBILITY_JAMO".equals(unicodeBlock)));
}

public static boolean isHangulOnly(String inputStr) {
    if (inputStr == null || "".equals(inputStr))
        return false;
    return isHangul(inputStr, true);
}

public static boolean isNumberOnly(String digit) {
    if (digit == null)
        return false;
    return isDigit(digit);
}

public static boolean isAlphabetNumberOnly(String str) {
    if (str == null)
        return false;
    return str.matches("[0-9a-zA-Z]+$");
}

public static String lpad(String orgStr, char appender, int length) {
    if (orgStr == null)
        return null;
    int orgLen = orgStr.length();
    if (orgLen >= length)
        return orgStr;
    StringBuffer sb = new StringBuffer();
    for (int i = 0; i < length - orgLen; i++) {
        sb.append(appender);
    }
    sb.append(orgStr);
    return sb.toString();
}

public static String rpad(String str, char appender, int length) {
    if (str == null)
        return null;
    int orgLen = str.length();
    if (orgLen >= length)
        return str;
    StringBuffer sb = new StringBuffer();
    sb.append(str);
    for (int i = 0; i < length - orgLen; i++) {
        sb.append(appender);
    }
    return sb.toString();
}

```

```

public static String ltrim(String str) {
    if (str == null)
        return null;
    return str.replaceFirst("^\\s+", "");
}

public static String rtrim(String str) {
    if (str == null)
        return null;
    return str.replaceFirst("\\s+$", "");
}

public static String left(String str, int len) {
    if (str == null)
        return null;
    if (str.length() <= len)
        return str;
    return str.substring(0, len);
}

public static String right(String str, int len) {
    if (str == null)
        return null;
    if (str.length() <= len)
        return str;
    return str.substring(str.length() - len);
}

public static int parseInt(String inputValue) {
    return Integer.parseInt(inputValue.trim());
}

public static long parseLong(String inputValue) {
    return Long.parseLong(inputValue.trim());
}

public static float parseFloat(String inputValue) {
    return Float.parseFloat(inputValue.trim());
}

public static double parseDouble(String inputValue) {
    return Double.parseDouble(inputValue.trim());
}

public static byte parseByte(String inputValue) {
    return Byte.parseByte(inputValue.trim());
}

public static BigDecimal parseBigDecimal(String inputValue) {
    return new BigDecimal(inputValue.trim());
}

public static int[] toIntArray(String strNum) {
    if (strNum == null)
        return null;
    int strNumLength = strNum.length();
    int[] arr = new int[strNumLength];
    for (int i = 0; i < strNumLength; i++) {
        arr[i] = Character.digit(strNum.charAt(i), 10);
    }
    return arr;
}

```

```

}

public static String nullToBlank(String inputValue) {
    if (inputValue == null)
        return "";
    return inputValue;
}

public static String join(String prefix, String suffix, String[] msg, String delim) {
    if (msg == null || msg.length == 0)
        return null;
    int msgLen = msg.length;
    StringBuffer sb = new StringBuffer(1000);
    for (int i = 0; i < msgLen; i++) {
        if (i > 0)
            sb.append(delim);
        sb.append(msg[i]);
    }
    if (prefix != null) {
        sb.insert(0, delim);
        sb.insert(0, prefix);
    }
    if (suffix != null) {
        sb.append(delim);
        sb.append(suffix);
    }
    return sb.toString();
}

public static String join(String[] msg, String delim) {
    return join((String)null, (String)null, msg, delim);
}

public static String checkNull(String param) {
    if (param == null)
        return "";
    return param;
}

public static String convertAtoB(String input, String condition, String output) {
    if (input == null)
        return (condition == null) ? output : input;
    return input.equals(condition) ? output : input;
}

public static Object nvl(Object obj, Object value) {
    return (obj == null) ? value : obj;
}

public static String nvl(String str, String value) {
    return (str == null) ? value : str;
}

public static String nvl(String str) {
    return (str == null) ? "" : str;
}

public static String lpad(String orgStr, int length, char appender) {
    if (orgStr == null)
        return null;
    int orgLen = orgStr.length();
    if (orgLen >= length)

```

```

        return orgStr;
    StringBuffer sb = new StringBuffer();
    for (int i = 0; i < length - orgLen; i++) {
        sb.append(appender);
    }
    sb.append(orgStr);
    return sb.toString();
}

public static final boolean isNull(String str) {
    return !(str != null && !str.trim().equals(""));
}

public static String repeat(int repeat, String mask) {
    int j = 0;
    int k = 0;
    if (mask == null)
        return null;
    if (repeat <= 0)
        return "";
    int inputLength = mask.length();
    if (repeat == 1 || inputLength == 0)
        return mask;
    if (inputLength == 1 && repeat <= 8192) {
        char[] arrayOfChar = new char[repeat];
        int arrayOfCharLength = arrayOfChar.length;
        for (int m = 0; m < arrayOfCharLength; m++) {
            arrayOfChar[m] = mask.charAt(0);
        }
        return new String(arrayOfChar);
    }
    int outputLength = inputLength * repeat;
    switch (inputLength) {
        case 1:
            char ch = mask.charAt(0);
            char output1[] = new char[outputLength];
            for (j = repeat - 1; j >= 0; j--) {
                output1[j] = ch;
            }
            return new String(output1);
        case 2:
            char ch0 = mask.charAt(0);
            char ch1 = mask.charAt(1);
            char output2[] = new char[outputLength];
            for (k = repeat * 2 - 2; k >= 0; k -= 2) {
                output2[k] = ch0;
                output2[k + 1] = ch1;
            }
            return new String(output2);
        default:
            break;
    }
    StringBuffer buf = new StringBuffer(outputLength);
    for (int i = 0; i < repeat; i++) {
        buf.append(mask);
    }
    return buf.toString();
}

public static String repeat(String str, String mask) {
    return repeat(mask, (str != null) ? str.length() : 0);
}

```

```

public static String repeat(Integer str, String mask) {
    return repeat(mask, (str != null) ? String.valueOf(str).length() : 0);
}

public static String repeat(long str, String mask) {
    return repeat(mask, String.valueOf(str).length());
}

public static String repeat(Long str, String mask) {
    return repeat(mask, (str != null) ? String.valueOf(str).length() : 0);
}

public static String repeat(float str, String mask) {
    return repeat(mask, String.valueOf(str).length());
}

public static String repeat(Float str, String mask) {
    return repeat(mask, (str != null) ? String.valueOf(str).length() : 0);
}

public static String repeat(double str, String mask) {
    return repeat(mask, String.valueOf(str).length());
}

public static String repeat(Double str, String mask) {
    return repeat(mask, (str != null) ? String.valueOf(str).length() : 0);
}

public static String repeat(short str, String mask) {
    return repeat(mask, String.valueOf(str).length());
}

public static String repeat(Short str, String mask) {
    return repeat(mask, (str != null) ? String.valueOf(str).length() : 0);
}

public static String repeat(BigDecimal str, String mask) {
    return repeat(mask, (str != null) ? str.toPlainString().length() : 0);
}

public static String repeat(BigInteger str, String mask) {
    return repeat(mask, (str != null) ? str.toString().length() : 0);
}

public static String repeat(String mask, int repeat) {
    int j = 0;
    int k = 0;
    if (mask == null)
        return null;
    if (repeat <= 0)
        return "";
    int inputLength = mask.length();
    if (repeat == 1 || inputLength == 0)
        return mask;
    if (inputLength == 1 && repeat <= 8192)
        return padding(repeat, mask.charAt(0));
    int outputLength = inputLength * repeat;
    switch (inputLength) {
        case 1:
            char ch = mask.charAt(0);
            char output1[] = new char[outputLength];

```